

Implementing session timeout in Next.js involves a combination of **server-side session management** (to enforce security) and **client-side activity tracking** (to provide a user-friendly experience, such as a warning modal).  Clerk +2

1. Server-Side (Next.js API Routes/Middleware)

The actual session expiration should be primarily managed on the server side for security, typically through cookies or JWTs that have built-in expiration times. Authentication libraries like NextAuth.js (Auth.js) or Auth0 handle this automatically.  DEV Community +4

- **Configure Expiration:** Use the configuration options of your authentication library to set the `idleTimeout` and `absoluteDuration` (hard cap) for sessions.
- **Check Expiration:** On every protected API request or page load, the server/middleware should validate the session's status. If expired, it should return a 401 Unauthorized status, which your client-side logic can then use to force a logout.  Stack Overflow +2

2. Client-Side (React Components)

Client-side logic enhances user experience by actively monitoring user inactivity and providing warnings, without relying solely on passive server checks. The `react-idle-timer` library is recommended for this purpose.  Bit.dev +3

A. Install the library

bash

```
npm install react-idle-timer
```

B. Create a `SessionTimeout` component

This component wraps your application and uses `useIdleTimer` (or `IdleTimer` in class components) to track user activity.  Codementor +1

jsx

```
// components/SessionTimeout.jsx
import React, { useRef, useState } from 'react';
import { IdleTimer } from 'react-idle-timer';

const SessionTimeout = ({ isAuthenticated, logOut }) => {
  const [timeoutModalOpen, setTimeoutModalOpen] = useState(false);
  // Configure timeouts (15 min inactivity, 2 min warning)
  const idleTime = 1000 * 60 * 15;
```

```
const warnTime = 1000 * 60 * 2;

const handleLogout = () => {
  // Call logout function (e.g., signOut() from NextAuth)
  logout();
};

const onIdle = () => {
  // Show warning modal and initiate countdown for logout
  if (isAuthenticated) setTimeoutModalOpen(true);
};

// Reset session when user acts
const onActive = () => {
  if (timeoutModalOpen) setTimeoutModalOpen(false);
};

return (
  <IdleTimer
    onActive={onActive}
    onIdle={onIdle}
    timeout={idleTime}
  />
  // Integrate a Modal component here to show warnings and
  // allow users to log out or continue.
);
};
```

C. Integrate into your app layout

Wrap your app or layout component with `SessionTimeout`, passing the authentication state and logout function.  Codementor +1

jsx



```
// app/layout.jsx
import SessionTimeout from '../components/SessionTimeout';

export default function RootLayout({ children }) {
  // Replace with real auth state/methods
  const isAuthenticated = true;
  const logout = () => { /* ... */ };

  return (
```

```
<html>
  <body>
    <SessionTimeout isAuthenticated={isAuthenticated} logOut={logout} />
    {children}
  </body>
</html>
);
}
```

Summary of Best Practices

- **Server-Side Security:** Always verify session validity on the server for secure API/route access.
- **Client-Side UX:** Utilize `react-idle-timer` to detect inactivity and provide user warnings.
- **Resource Management:** Properly clear `setTimeout` and `setInterval` to prevent memory leaks.

 Stack Overflow +4